OOP Notes

Clases y Objetos

Llamamos objeto a toda variable que tiene como tipo de dato el de una clase, y llamamos clase a una estructura que agrupa datos y que contiene funcionalidades para manipularlos. Acá abro un pequeño paréntesis ya que este concepto si lo logré dominar bien. Abordando el tema de las clases desde un ámbito mucho más práctico, intuitivamente las podemos pensar como un molde, una estructura que nos va a permitir modelar cosas del mundo real, son un abstracción, un mini mundo; esto se logra analizando el objeto, extrayendo sus características esenciales, y como no, las acciones que puede realizar teniendo esas características.

Pensemos por ejemplo en una fecha, sí, en una fecha. Las fechas a simple vista nos pueden parecer diferentes unas de las otras ya que son distintas (a no ser que sea la misma fecha, obviamente), pero realmente todas comparten unas características en común, el día, el mes, y el año, y estas características las llamaremos atributos, los cuales dentro del lenguaje nos permiten almacenar datos sobre cada fecha, aunque todas compartan la misma característica realmente cada una tiene un día diferente por ejemplo, o un mes, o un año; esa diferencia en el valor de sus atributos es lo que las hace fechas distintas entre sí. Ya a los objetos los llamamos Fechas, y cada objeto es una instancia de esa clase, es independiente de los demás, pero sigue siendo una fecha, es una realización de esa estructura, es producto de esta, es del mismo tipo (Fecha), pero como tal ya es un objeto creado a partir de ese molde.

Los métodos

Los métodos son bloques de código encapsulado con nombre que realizan una acción específica, y esta puede ser repetida cuantas veces se llame a la función.

Los métodos son fundamentales porque son el medio principal para interactuar/trabajar con objetos que ya están instanciados, y lo mejor de todo es que ofrecen un nivel de protección e integridad para los valores de los atributos del objeto (datos), ya que accede y trabaja con ellos de forma interna (incluso los que son privados), sin que el cliente pueda verlos o modificarlos, lo cuál añade una capa más de fiabilidad. Estos métodos son como un domiciliario o alguien que te hace un favor, tu lo llamas, le dices que comprar, le das los parámetros y el va, hace su acción y te entrega un resultado, o nada si no debe entregarte algo.

Además, nos proveen modularidad, haciendo que las responsabilidades se repartan más, y permiten ahorrar muchas líneas de código; también pueden trabajar de forma diferente incluso bajo el mismo nombre, por ejemplo un domiciliario puede traerte comida china o italiana, y tu le puedes dar dinero en efectivo o pagarle con tarjeta de crédito; y también el domiciliario puede tomar rutas distintas para llegar al restaurante, estos dos conceptos se llaman sobrecarga y sobrescritura, que modifican tanto los parámetros que puede recibir un método como su implementación interna.

Los métodos como tal pueden ser implementados por tres entes, las clases, las clases abstractas y las interfaces, aunque las interfaces y las clases abstractas no los implementan, solamente los declaran, a esto se le llama métodos no concretos o abstractos

- Una pregunta para más adelante ¿En las interfaces solo se definen métodos públicos?: Respuesta, **sí**, tradicionalmente solo se definen métodos públicos y abstractos, pero con la llegada del java 9 se pueden implementar métodos privados estáticos; aunque el propósito de las interfaces es establecer un contrato de comportamientos comunes o partes de un API que se debe exponer

Para acceder o modificar directamente (bajo ciertas validaciones cabe aclarar) los atributos de un objeto se usan los getters y los setters. Los getters permiten obtener el valor del atributo y los setters modificarlo. En C# existen las properties, que nos ayudan a agrupar en una misma expresión un atributo y las reglas de acceso a este.

Herencia y sobreescritura de métodos

En java, la herencia es un tema de suma importancia, ya que permite crear clases en función de otras, a partir de un sistema jerárquico en el que no se permite herencia o multiplicidad de relaciones de este tipo de manera ascendente. Una clase al ser creada tiene sus propios atributos y métodos, y si hereda, también posee los atributos y los métodos de la clase de la que heredó, pero eso no significa que los pueda ver o modificar. Además, la herencia en Java es transitiva, es decir, tengo por ejemplo tres clases A, B, y C; A se crea y B hereda de A, y luego C hereda de B, por lo tanto C hereda también de A.

En Java, absolutamente todo lo que no sea un tipo de dato primitivo es un objeto (el cuál es creado a partir de los tdp), y ese objeto hereda de una clase padre (por defecto), la clase base de todas las clases, la clase Object, y como no, esta clase tiene ciertos métodos que una clase debe implementar opcionalmente para mejorar su funcionamiento. Los métodos más conocidos son el toString(), equals() y el hashCode(), aunque hay otros cuantos. El hecho de que en java exista una clase llamada object hace referencia a que el objeto que se crea y hereda de esta es un **objeto u object**; cualquier objeto en java si o si debe cumplir con ciertas funcionalidades básicas que le permitan ser un objeto bien declarado dentro del lenguaje

llamamos sobreescritura a una nueva implementación del mismo método que heredamos de una clase padre

Reglas para la sobreescritura de un método

- Firma menos menos o igualmente restrictiva que la declarada inicialmente: Parámetros, nombre, tipo de retorno
- El tipo de retorno puede ser covariante, es decir, pueden ser tipos de datos compatibles, es decir, que sea subclase del tipo original
- La visibilidad no puede ser más restrictiva:

Clase base	Clase derivada	¿Válido?
`public`	`public`	✅
`protected`	`protected` o `public`	✅

`public`	`protected` o `private`	❌
`protected`	`private`	❌

Supongamos que tenemos una clase Persona y una clase Empleado, evidentemente la clase empleado hereda de persona, ya que pues un empleado es una persona, y a su vez la clase Persona hereda de Object, por lo que empleado también es un Object. En general se dice que cuando una clase hereda de otra se dice que **es un "otro"**

La clase object tiene ya implementados algunos métodos que se requieren para que teóricamente un objeto funcione de cualquier manera. Al ser Object una clase concreta, estos métodos se heredan y opcionalmente se sobrescriben según las necesidades de la clase en particular

toString();

Su propósito principal es devolver el objeto para el cuál lo estamos definiendo, en un formato de cadena de texto, esto para proveer información sobre el estado interno o algún detalle que facilite su lectura. Si no lo sobrescribimos simplemente devuelve el nombre de la clase con una dirección de memoria en hexadecimal

equals()

Su propósito es poder comparar si dos objetos del mismo tipo son iguales, retorna un booleano true si son iguales, y si no false. Si no se sobrescribe compara únicamente si los objetos se ubican en la misma posición en memoria; esto puede representar un problema si nuestro objeto de comparación son sus atributos, ya que puede que dos instancias se ubique en distintos lugares pero tener atributos iguales (eso las hace iguales), por lo tanto habría que comparar también ese caso.

Definir e instanciar objetos

Para crear como tal un objeto no basta solo con declararlo, es decir, poner su tipo de dato y su nombre, hay que instanciarlo. Los objetos funcionan mediante referencias, que son por decirlo así punteros a direcciones de memoria, cuando únicamente declaramos el objeto este apunta a una referencia nula

```
Objec object; // acá unicamente se declara, la referencia apu
nta a null, riesgo de NULLPOINTEREXCEPTION
object = new Object(); //creamos el objeto, y este apunta a u
na dirección en memoria vacía

//Tranquilamente podríamos haber definido el objeto por compl
eto, declarándolo e instanciándolo
Object object = new Object();
```

El método constructor

El constructor es un método especial través del cuál podemos crear los objetos. El Java nos da un método constructor por defecto llamado “constructor nulo”, este no recibe parámetros, simplemente crea una instancia vacía. Al momento de crear la instancia de un objeto como tal vemos dos palabras, “new” y “Nombre()”; la palabra reservada new está reservada en memoria y nos sirve para “avisar” que vamos a llamar el método.

El propósito del método constructor es la creación de un objeto mediante la inicialización de sus atributos, y garantizando que la instancia se cree en un estado válido; en el constructor por defecto o nulo, se inicializan las variables de instancia con los valores por defecto que el lenguaje establece, 0 para valores numéricos, false para booleanos y Null para objetos. Es genial ya que nos permite dejar de lado los setters para poder inicializar las variables, y que más bien solo halla un punto de entrada en el cuál los valores de los atributos se ingresen, para así garantizar más encapsulamiento. También permite la sobrecarga para poder tener múltiples maneras de instanciar los objetos.

Aunque más adelante, cuando se vea la cuestión de encadenamiento de métodos, veremos que no es para nada malo usar los setters para inicializar variables o atributos de instancia, y es una muy buena práctica en distintos tipos de arquitectura usando la técnica anteriormente mencionada para mejorar la legibilidad del código, siempre y cuando el atributo o el objeto mismo no deban ser inmutables.

Características básicas del constructor:

- Mismo nombre de la clase
- No tiene tipo de retorno
- Casi siempre con modificador de acceso público (public), más adelante veremos casos de uso con los demás
- No se implementa funcionalidad dentro de este, es importante no mezclar las etapas de vida de un objeto

Ejercicio Integrador: Realizar un pequeño sistema bancario con cuentas, depósitos, retiros y usuarios

Nomenclaturas usadas para Java

- PascalCase: Clases
- camelCase: Métodos, atributos, variables locales o de instancia
- UPPER_SNAKE_CASE: constantes

Sobrecarga

El concepto de sobrecarga consiste en que un mismo método pueda recibir más de una combinación de tipos y/o cantidades de argumentos. Es muy importante no confundir el concepto de sobrecarga con el de sobreescritura. Como bien sabemos la sobreescritura está asociada con los métodos heredados de una clase padre (o una interfaz, pero eso lo veremos más adelante), estamos volviendo a programar el mismo método pero desde nuestra clase hija, respetando los parámetros que estaban inicialmente implementados en la clase padre.

Encapsulamiento

El encapsulamiento consiste en la aplicación de ciertas herramientas que nos sirven como base para el ocultamiento y protección de los datos, haciendo que estos se agrupen de forma segura con las funcionalidades abstraídas

---

Visibilidad de atributos y métodos

Nos encontrábamos hablando a cerca del concepto de encapsulamiento, pero es que los niveles de visibilidad nos ayudan a implementar el encapsulamiento de

forma adecuada. Existen en Java 4 niveles de visibilidad principales

- public (acceso ilimitado desde cualquier lugar): si algún atributo o método se encuentra declarado como public es porque cualquier clase externa de la que nos encontramos trabajando puede accederlo, por ejemplo en el caso de los atributos pueden cambiar o ver su valor, y para los métodos, estos pueden ser invocados desde cualquier parte cuantas veces lo desee el usuario
- private (acceso solo desde la clase): con el modificador de acceso private ya la cosa cambia, el atributo o método que sea declarado de esta manera solo puede ser usado dentro de la clase que lo implementa, en ninguna más. Se estiliza que los atributos de una clase sean private y los métodos que sirven para interactuar con el objeto se declaran public, no obstante, si un método de estos debe ser necesariamente dividido en métodos más pequeños (auxiliares), estos deben ser declarados como privados para no confundir al usuario con funcionalidades que no tendría por qué estar viendo
- protected: este modificador de acceso es un poco más complejo porque tiene varios contextos en donde las clases pueden acceder a los atributos y métodos declarados con él. El primero sería si las clases se encuentran dentro del mismo package (sin necesidad de tener una cadena de herencia), la segunda es si las clases se encuentran o no en el mismo package pero sí tienen una relación de herencia. Las clases que no cumplan las condiciones anteriormente mencionadas no pueden acceder a los atributos o métodos declarados como protected
- Default o Private-Package este modificador de acceso es el que el Java implementa por defecto cuando no declaramos explícitamente un modificador de acceso, cuando no ponemos nada solo únicamente las clases que se encuentran en el mismo package pueden acceder a los atributos y métodos que “no tienen” modificador

NOTA: Los atributos que son declarados privados SI SON HEREDADOS, pero no pueden ser accedidos

📌 **Tabla resumen:**

Modificador	Misma clase	Mismo paquete	Subclase (otro paquete)	Otro paquete
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌
(default)	✅	✅	❌	❌
`private`	✅	❌	❌	❌

Los Packages

Los paquetes constituyen un mecanismo para agrupar/organizar nuestras clases bajo un criterio determinado (el criterio es relativo, según lo que nosotros queramos), nos ayudan a construir los namespaces (espacios de nombre), los cuales permiten tener clases con un mismo nombre siempre y cuando pertenezcan a paquetes distintos

Para **declarar** que una clase pertenece a cierto paquete, la primera línea de código que deberemos implementar será:

```
package "ruta.en.donde.está.la.clase";
```

y usualmente el compilador nos suele ser obligar a declararla cuando el proyecto se divide en mucho directorios.
Los packages nos permiten ir mucho más allá, ya que ayudan a manejar proyectos a gran escala y a reutilizar el código que ya escribimos para no tener que escribir las mismas funcionalidades/utilidades una y otra vez.

Funcionan realmente como una división lógica que nos permite agrupar nuestras clases y posteriormente importarlas o “llamarlas” y usar sus funcionalidades dentro de otros proyectos o clases que pertenezcan a un package distinto (pero con limitaciones, no todo es tan mágico)

La cuestión funciona de la siguiente manera: Nosotros tenemos nuestro proyecto ubicado en una carpeta tipo mi\proyecto\Ordenamiento y mi\proyecto\EDAS, y supongamos que queremos poder usar las funcionalidades que se encuentran en desarrollo dentro del otro, para eso acudiremos a los packages. A las clases que tengan los algoritmos de ordenamiento en la primera línea les debemos especificar a que package pertenecen por ejemplo, package

mi.proyecto.ordenamiento, y a las otras les ponemos package mi.proyecto.edas, y así tendremos ya nuestras clases asociadas y divididas por paquetes. Por ejemplo, ahora sí podríamos usar las funcionalidades de ordenamiento en la carpeta de edas, para ordenar una lista de elementos o lo que sea que fuere un objeto ordenable.

Pero esto no solo se queda acá, también podemos importar esos packages en otros proyectos que no tengan la misma ruta, por ejemplo en uno que se llame proyecto\mi\empresa\"carpetas del proyecto..." y usarlos en ciertas partes del código que requieran funcionalidades que uno como programador ya ha diseñado y que no vale la pena volver a implementar. Para este caso en específico cambia un poco la manera de operar ya que tenemos varios caminos para poder usar esas funcionalidades diversas. El primero sería decirle a nuestro gestor de dependencias, ya sea gradle o maven, que queremos usar esas clases, de esa manera el irá a internet y la descargará para su uso; la siguiente alternativa es especificarle al classpath, la cuál es una variable de entorno, donde se encuentra nuestro archivo .class (el código compilado a bytecode de nuestra clase) o el .jar que contienen las funcionalidades deseadas, y el ya nos permitirá importarlo con total normalidad

→ la carpeta bin de un proyecto almacena todos los archivos compilados en bytecode para la posterior ejecución

→ el mecanismo para crear namespaces es el package

API

Es un conjunto de clases, métodos, reglas o funciones que un software expone para que otras aplicaciones puedan interactuar con el sin saber como este está implementado internamente

Recordemos que una interfaz es un medio por el cuál dos sistemas interactúan entre sí, es una conexión o frontera común entre ambos. También se puede ver como una conexión físico-lógica entre un usuario y un sistema de cómputo.

Los ADT

Entramos en el terreno de los ADTS, que básicamente son las clases/objetos en este contexto. El ADT como tal provee una abstracción de un ente del mundo real, el cuál tiene sus propios atributos/características y operaciones, estas operaciones pueden ser tanto internas (de el mismo), como externas (tienen una salida más allá de su propio mundo).
En este contexto existen dos roles:

- Cliente: el cliente hace uso del ADT a través de sus métodos públicos, estos métodos públicos tienen el rol de API, conectan el ADT con el cliente y permiten que este pues use sus funcionalidades respetando el encapsulamiento, ya que los detalles de implementación siempre se ocultan al cliente
- Implementador: este es el que decide como se va a representar el valor del ADT y decide los algoritmos que va a implementar para sus operaciones

---

Polimorfismo

El polimorfismo significa muchas formas. En el contexto de OOP, el polimorfismo se refiere a los múltiples comportamientos que puede tener una referencia dependiendo del tipo de objeto que esta contenga internamente

→ los objetos nunca dejan de reconocerse como miembros de una única clase, por tal motivo, independientemente del tipo de dato de la variable que los contenga, estos siempre van a seguir comportándose como se definió en la clase a la que pertenecen.

- Estático (en tiempo de compilación): se logra a través de la sobrecarga de métodos. Recordemos que la sobrecarga de métodos permite tener múltiples métodos con el mismo nombre pero con distintos tipos de parámetros (ya sea en cantidad o tipos de parámetros), y distintos tipos de datos de retorno. Acá el compilador elige cuál método elegir según los tipos de datos que se le pasaron al método.
- Dinámico (en tiempo de ejecución): Este se logra a partir de la sobreescritura de métodos heredados. Este ocurre cuando una clase hija sobreescribe un método que ya está definido en la clase padre (method overriding). Esto permite que una referencia del tipo de dato de la clase padre sea capaz de llamar los métodos necesarios dependiendo del tipo de dato que esta

contenga internamente. Por ejemplo un objeto tipo object contiene un objeto tipo fecha, cuando llamamos a toString() en tiempo de ejecución se llama al método que está sobreescrito en la clase fecha, y esto se decide en tiempo de ejecución. Por lo general este tipo de polimorfismo se usa para almacenar objetos del tipo de las subclases dentro de referencias de las superclase, y llamar comportamientos **genéricos,** es decir, que se heredaron y sobrescribieron

De este tema ya habrá otro inciso particular llamado **Static vs Dynamic Binding**

Imaginemos el polimorfismo como la interpretación de una palabra, por ejemplo correr. La palabra correr puede significar “correr un programa” o “correr en una carrera”, según el contexto se interpreta de la manera que se requiere. El polimorfismo es bello porque permite un acoplamiento sencillo de nuevos objetos en el sistema, solo basta que los anteriormente mencionados implementen los métodos correspondientes para poder pertenecer de manera válida a la jerarquía, y esto logra que los sistemas en general diseñados de esta manera sean fácilmente extensibles

Principio de sustitución de liskov

Un objeto de una subclase debe poder sustituir a un objeto de su superclase sin alterar el correcto funcionamiento del programa

Ejemplo con la clase animal y la clase perro: Un perro puede instanciarse en un tipo de dato de animal porque perro contiene y hereda todo que es un animal, y además añade las características de perro, este suple a animal sin alterar el funcionamiento del programa.

Constructores de subclases

Lo primero que se debe hacer a la hora de crear el constructor de una subclase, es llamar al constructor de la clase padre, si no hacemos esto explícitamente, el Java intentará llamar automáticamente el constructor nulo, pero si este no está definido (recordemos que este por defecto existe, pero si creamos un nuevo constructor lo tenemos que definir explícitamente) habrá un error de compilación.

- Los constructores no se heredan, hay que llamarlos explícitamente, esto permite que el objeto que se va crear se inicialice en un estado válido

¿Por qué sucede esto?

El problema surge porque precisamente la clase que hereda de la clase padre es un “tipo de dato del padre”, y por lo tanto este hereda algunos de sus atributos, y si los hereda los debe instanciar, por eso se llama al constructor. Recordemos que un objeto que está en función de una clase debe ser capaz de sustituir al objeto de la clase base y no alterar el funcionamiento del programa, es decir que al este heredar esos comportamientos debe definirlos también, ya que hacen parte de su identidad

¿Cómo llamamos al constructor de la clase padre?

con la referencia super(), esta llama al constructor de la clase padre, y le debemos pasar exactamente la combinación de atributos definida inicialmente en la clase base para su respectivo constructor. Un uso peculiar es que puede hacer referencia al objeto de la clase base, pudiendo así usar las funcionalidades de este, es decir, permite volver a un tipo de dato más genérico.

La referencia this

Así como la palabra reservada super hace referencia a el constructor de la clase base, la palabra reservada this hace referencia a los constructores de la propia clase. Por ejemplo:

```
public Fecha (int dia, int mes, int anio) {
    diaSemana = dia;
    mesDelanio = mes;
    anio = anio;

    }

public Fecha() {
    this(0,0,0); //llamamos al constructor fecha de la parte
de arriba
}
```

en este ejemplo tenemos dos constructores para la clase fecha. Hay uno que recibe parámetros y el otro no recibe parámetros; esa referencia this en el segundo, invoca al constructor que tiene exactamente los mismos parámetros (en este caso int), y los inicializa en 0, es como el constructor vacío, pero en vez de hacerlo el le pasa el trabajo a su hermano.

También esta palabra nos funciona para hacer referencia a nosotros mismos, al estado actual de la propia instancia del tipo de la clase, entonces gracias a ello se pueden diferenciar los atributos de la clase (las variables de instancia) de el nombre de alguna otra variable, o también retornar al objeto en sí

```
public Persona setNombre(String nombre) {
    this.nombre = nombre;
    return this; //para encadenamiento de métodos, lo veremos
más adelante
}
```

Abstracción

La abstracción consiste en que dado un sistema compuesto por entidades, cada una de estas debe ser representada de forma esencial y concreta. No consiste en omitir deliberadamente detalles o modelar el objeto de forma incompleta, sino más bien haciéndolo de forma resumida, de tal manera que se pueda separar de lo verdaderamente necesario de lo particular, ello con sus respectivos atributos o características, las funcionalidades que este expone para que otros interactúen con el, y las internas que este tiene pero que no le interesan a los demás. Por ejemplo, en el contexto académico, un estudiante posee notas, posee un promedio, posee materias que está viendo, pero también tiene un color de ojos, grupo de amigos, deportes favoritos; la clave está en saber que datos son realmente significativos para la solución y modelado de nuestro problema.

Realmente la abstracción es un concepto más teórico, y que sin embargo, tiene varias vías de aplicación dentro del lenguaje Java. Por ejemplo, las clases en si mismas son abstracciones de estado y comportamiento, ya que permiten modelar un objeto de acuerdo a sus rasgos esenciales y comportamientos principales; también, se ofrece como alternativa las clases abstractas y las interfaces (hablaremos más adelante de estas), las cuales proveen abstracciones de comportamiento, permiten modelar un objeto de manera que se respete un sistema jerárquico en donde se considera el qué y el cómo, como acciones distintas, permitiendo así generalizar el qué (centrándose en los rasgos esenciales) y particularizando el como (cada clase que hereda de estas decide como hacer lo que hace). Cuando el programador interactúa con un objeto, lo hace por medio de las abstracciones operativas o de comportamiento que este expone mediante una interfaz para ser utilizado, sin embargo, el que codifica (en este caso asumiendo el rol de cliente) no debe preocuparse por detalles complejos que emergen de la implementación del objeto, sino que estos lo usan por medio de lo que ellos exponen, a esto se le llama encapsulamiento, y no debe confundirse con la abstracción.
La abstracción se preocupa por definir que detalles internos se implementarán, el encapsulamiento se preocupa por brindar un medio seguro para interactuar con ellos, sin que estos se vean totalmente expuestos y afectando la integridad del objeto; el encapsulamiento y la abstracción bien implementados evitan que objetos dependan de otros de forma innecesaria, es decir, que hayan acoplamientos sin sentido, o que la integridad misma de un objeto se vea comprometida al exponerse tanto

En los diseños orientados a objetos, la abstracción se mide por la capacidad de representar un concepto de forma coherente sin la necesidad de mezclar responsabilidades ajenas a el. Un diseño con mala abstracción implica que hayan acoplamientos innecesarios, que objetos puedan modificar de forma directa el estado interno de otro, que los métodos requieran demasiado conocimiento del sistema para ser utilizados, etc. Mientras que un buen diseño se basa en lograr una gran combinación entre la abstracción y el encapsulamiento, que haya relaciones con cohesión entre objetos, que se logre evitar que objetos se compongan de otros, y que cada uno sea independiente de los demás, que pueda controlar de forma interna su estado, exponiendo un API adecuada para ello.
A un nivel más intuitivo, la abstracción puede imaginarse como un proceso

análogo al que realiza un cartógrafo. Un mapa no es una copia fiel del territorio, sino una representación selectiva que omite la mayor parte de los detalles: no muestra cada árbol, cada roca o cada elevación menor. En su lugar, conserva solo la información que es necesaria para la navegación. De la misma forma, una clase abstracta representa el “mapa conceptual” de un conjunto de objetos concretos. No pretende mostrar cómo funcionan internamente ni detallar cada variación posible, sino ofrecer una vista conceptual que permita trabajar con ellos sin perderse en la complejidad del territorio real. El valor de la abstracción radica precisamente en esa simplificación inteligente: mostrar solo lo que importa pensando únicamente en los detalles que resuelven el problema.

Clases Abstractas

Una clase abstracta es una clase que contiene método abstractos, los cuales son métodos que no pueden ser desarrollados por falta de información a cerca del objeto. Un ejemplo muy común es el de las figuras geométricas; nadie puede negar que a todas las figuras geométricas les podemos calcular el área, sin embargo el cálculo del área depende de que figura geométrica es, por lo tanto no podríamos resolver el método calcularArea() ya que nos falta la información sobre que figura es. Por lo tanto, estos métodos abstractos deberán ser desarrollados una vez se sepa que figura es, cuando la información correspondiente se encuentre disponible. Las clases abstractas se declaran como:

```
"modificador de acceso" abstract class NombreDeLaClase
```

En el caso de la figura geométrica sería

```
public abstract class FiguraGeometrica {
    public abstract double calcularArea(); // no desarrollamo
s el método
```

```
}
```

como los métodos de las clases abstractas no están desarrollados, estas no pueden ser instanciadas

Sin embargo, eso quiere decir que todos sus métodos deben ser abstractos? No, las clases abstractas también pueden contener métodos concretos que se heredan y que pueden ser opcionalmente sobrescritos, a diferencia de los abstractos

- los métodos abstractos no tienen bloque de código implementado, no llevan llaves
- los métodos abstractos solo pueden ubicarse en clases abstractas, no en las normales
- cuando una clase hereda de una clase abstracta debe implementar todos sus métodos o si no esta deberá declararse abstracta también

Todo método abstracto debe ir en una clase abstracta, pero una clase abstracta puede contener métodos concretos o normales también. Esto ayuda a definir un comportamiento común para las subclases, y que algunos métodos queden como “pendientes por implementar”. El poder de la abstracción y el polimorfismo puede llegar a ser brutal ya que podemos llegar a invocar esos métodos que son abstractos en una variable del tipo de dato de la clase abstracta, pero que esta internamente contenga una referencia a una de sus clases hijas las cuales poseen el método concretado. Con clases normales se heredan comportamientos ya definidos, sin embargo en las abstractas no se define y estas **obligan** a que cada subclase lo defina. Las clases abstractas permiten separar comportamientos específicos y comunes para las subclases, lo que permite tratar de mejor manera objetos de distintos tipos bajo un mismo tipo base (mejor polimorfismo). Usualmente estas suelen utilizarse cuando se quiere definir jerarquías de herencia que por supuesto deben de tener una conexión semántica, en donde hayan cierto tipo de comportamientos comunes (a diferencia de las interfaces 😉).

Constructores de clases abstractas

A pesar de que las clases abstractas no puedan ser instanciadas puesto que le faltan métodos por concretar, esta si puede tener un constructor. La clase abstracta es una especie de mezcla entre métodos no definidos y métodos concretos, y esta también permite definir comportamientos comunes para las subclases, y por lo tanto también se pueden tener atributos comunes para cada subclase. Por ejemplo, todas las figuras geométricas tienen un nombre, sin embargo la operación de calcular su área depende de que figura es; entonces el constructor de la clase abstracta nos sirve para poder forzar a que en la subclase se de un valor para ese atributo heredado y que la instancia de la figura geométrica esté en un estado válido.

Interfaces

La definición técnica de interfaz radica en que estas son un medio de interacción entre dos sistemas, o dicho de una mejor manera, son un conjunto de funcionalidades, métodos o propiedades que un software u objeto expone para que puedan interactuar con el de forma controlada. Hasta acá nada que no sepamos, sin embargo, en Java, puede que varios objetos que sean dispares, es decir, que no estén necesariamente relacionados mediante herencia, tengan comportamientos que se comparten en específico, y que obligatoriamente deben implementar, esta es la funcionalidad de las interfaces en Java.

Son un contrato que firman las clases, en el cuál aseguran que deben concretar los métodos declarados por la interfaz que desean implementar, y si no lo hacen deberán ser declaradas como clases abstractas (esto ya va más en cuestiones de diseño). Las interfaces permiten entonces que este tipo de objetos se puedan comportar de manera polimórfica mediante el compartimiento de métodos y constantes comunes entre estos.

Buenas prácticas con interfaces en programación:

- Declarar los métodos sin el public abstract, ya que por defecto en una interfaz se consideran así
- Declarar las constantes sin el static final ya que por defecto estas son así en una interfaz

- Nombramiento genérico para los métodos

Más sobre las instancias

Los objetos son instancias de una clase porque estos poseen distintas combinaciones en los valores de los atributos que define la clase. Las frases como “crear un objeto” o “instanciar la clase” se usan realmente como sinónimos. Las instancias son una realización, una pequeña creación a partir de una clase.

Las variables de instancia:

Decimos que los atributos de una clase son variables de instancia ya que cada instancia de la clase tiene sus propios valores para estos. Sin embargo, no todas las variables de instancia son atributos, ya que a veces estos no necesariamente describen algo del objeto en particular, sino que pueden ser por ejemplo recursos para la programación, no obstante, cada objeto tendrá un valor único para cada uno de estos, sin necesidad de ser considerado como un atributo en sí.

Las variables de la clase:

Así como cada objeto o instancia de la clase tiene valores propios para sus variables de instancia, la clase puede tener un valores propios, es decir, que sea común a todos los objetos y que estos lo compartan. Esto se logra gracias al modificador static, y gracias a ello todas las instancias de la clase pueden acceder a ese mismo valor, sin embargo este no es propio de una en particular sino de la clase en sí.

Constantes de instancia y de la clase

Son variables de la clase o de instancia que poseen un valor inmutable, que nunca va a cambiar luego de ser inicializado, para ello usamos la palabra reservada final. Recordemos que la static es para hacer la variable de la clase

```
public static final double constante = 3.1415; //se definen c
omo static final
public final String nombre = "Paulina Guiral Ortega";
```

```
//ya esos valores no pueden cambiar, se debe crear otra varia
ble nueva
```

El Garbage Collector

El gc es un proceso que está constantemente buscando bloques de memoria des referenciados para liberarlos y que así otros procesos los puedan ocupar. Cuando nosotros instanciamos un objeto lo que hacemos es tomar un pedazo de memoria dinámicamente, y el objeto sirve para referenciar ese lugar de memoria que está asignado. En lenguajes como C o Pascal nosotros luego de usar la memoria dinámicamente debemos liberarla de forma manual para que otros procesos puedan usarla, sin embargo en java no debemos preocuparnos por eso. Hace muchos años el gc antes de eliminar a un objeto primero validaba si este tenía declarado un método llamado finalize(), este usualmente permitía ejecutar operaciones de liberación recursos antes de que el objeto en cuestión fuese totalmente eliminado, sin embargo dicho método quedó depreciado por motivos de peso ya que era muy inseguro, no se tenía certeza de cuando se invocaría, podía resucitar el objeto si había una nueva referencia a este y lograba causar dolores de cabeza para la liberación de recursos, y si ocurría una excepción en este la JVM simplemente la ignoraba; por tales motivos fue retirado y se considera una mala práctica usarlo. Para liberar recursos lo mejor sería usar try-with-resources.

Métodos finales

son métodos declarados con la palabra reservada `final` los cuales ayudan a que estas funciones no puedan ser sobrescritas, más sí heredadas. Un método declarado en una superclase como private es implícitamente final también, y lo mismo con uno static. Los métodos finales si pueden ser sobrecargados, incluso en subclases.

Métodos de la clase

Los métodos estáticos son conocidos como los métodos de la clase, ya que no tenemos que instanciarla para poder llamarlos. Podemos diferenciarlos del resto porque su valor de retorno únicamente se encuentra en función de los parámetros que este recibe y no tiene la necesidad ni la posibilidad de acceder a ninguna variable de instancia de la clase.

Clases Utilitarias:

De los métodos estáticos o utilitarios podemos derivar algo muy importante. Podemos considerar una clase utilitaria cuando la mayoría de sus métodos son estáticos, ya que estos no requieren acceso a ninguna variable de instancia, y por lo tanto solo nos sirven para realizar acciones en función de los parámetros que estos tengan. Las llamamos así porque usualmente cuando se suelen agrupar muchos métodos estáticos es porque se quiere proveer cierto tipo de funcionalidad.

Ciclo de vida de los objetos

En Java, el ciclo de vida de un objeto se refiere al conjunto de estados por los que puede pasar este, desde el momento de su creación hasta su eliminación y liberación de espacio en memoria gracias al recolector de basura. Es un proceso fundamentado en el modelo de memoria, el sistema de clases, el proceso de inicialización, y el garbage collector; gracias a ello se puede concluir que el objeto no es simplemente un espacio en memoria, sino una entidad que está sometida a reglas precisas que determinan su funcionalidad y continuidad a través del tiempo.
Este ciclo consta de cuatro pasos: la solicitud de creación del objeto, la construcción e inicialización, el periodo de vida útil y el proceso de selección y recolección por parte del garbage collector. Realmente este ciclo sucede de forma continua en el tiempo y muchas veces suele pasar desapercibido, pero conocer a detalle como trabaja cada etapa nos vuelve mejores como programadores enfocados en Java, ya que nos ayuda a comprender por qué este lenguaje a diferencia por ejemplo de C o C++ evita errores comunes relacionados con el manejo **manual** de la memoria y los recursos, y nos permite analizar los efectos de las inicializaciones y las referencias perdidas. Además, para la

ingeniería de software conviene analizar este ciclo para no mezclar responsabilidades o realizar acoplamientos innecesarios, por ejemplo que dentro del constructor haya funcionalidades de la etapa de construcción y de vida útil refleja un mal diseño con poca escalabilidad.

Solicitud de creación de un objeto:

La solicitud de creación del objeto está asociada a la declaración de la palabra reservada **new.** En este punto, cuando hacemos uso de dicha palabra, estamos ordenando a la JVM que reserve y limpie un espacio en memoria dentro del heap, el cuál tendrá un tamaño acorde con los atributos o características del objeto a crear, sin embargo, este espacio en memoria no tiene valores almacenados, puesto que aún no se invocó el constructor o se inicializaron los valores. Por defecto los atributos que son numéricos van en 0, los booleanos van con false, y null para las referencias.

El heap:

La memoria HEAP es un fragmento de la memoria que se usa para almacenar objetos en tiempo de ejecución, donde se almacenan la mayoría de datos dinámicos de una aplicación en java. Cuando se crea un objeto este se almacena en el heap.

Construcción e inicialización del objeto:

Este es un proceso por capas que garantiza la creación de cualquier objeto en un estado válido. En primer lugar, si el objeto sigue una cadena de herencia, la JVM se encargará de llamar el constructor del nivel de abstracción más alto, siguiendo con los niveles más bajos para garantizar que cada atributo se inicialice con valores válidos; posteriormente la JVM ejecuta los bloques de inicialización y asigna los valores explícitamente declarados para el objeto, y luego esta se encarga de ejecutar el cuerpo del constructor invocado, el cuál se encarga de establecer y validar las condiciones particulares del objeto recién creado. En este punto final, se supone que el objeto ya es totalmente funcional y está listo para ser usado

---

Periodo de vida útil:

En este punto el objeto ya existe dentro del heap, y este puede ser referenciado por variables locales. Definir cuando y por qué el objeto está en un periodo de vida útil depende de muchos aspectos, pero mientras este se encuentre activo podrá interactuar con otros objetos mediante sus métodos, podrá ser pasado como argumento, podría ser almacenado en colecciones y también cambiar su estado interno; sin embargo cuando el objeto sea desreferenciado, será vistopor el garbage collector para ser eliminado, la existencia de un objeto no depende de su usabilidad sino de su accesibilidad, si ese bloque de memoria dentro del heap ya no es accesible por ningún medio se dice entonces que el objeto está en un estado de “no alcanzable pero no recolectado”, y este se encuentra por lo tanto a la espera de que el GBC lo elimine

Desreferenciado significa que ya no existe ningún tipo de referencia o puntero que lleve al objeto en cuestión

```
//Ejemplo de el ciclo de vida
Puerta puerta = new Puerta(); // -> solicitamos la creación d
el objeto y luego el constructor

// hace su trabajo para crearlo en un estado válido
//---- PERIODO DE VIDA ÚTIL------
puerta.abrirPuerta();
puerta.cerrarPuerta();
puerta.cambiarPerilla()
//--------------------------------
puerta = new Reja(); //asumiendo que en el diseño las rejas s
on un tipo de puerta más específico
//acá ya hubo una desreferenciación para la puerta de las pri
meras líneas, el gc se encargará
// de liberar la memoria en el heap
```

Periodo de recolección:

Como sabemos, el GBC es un proceso que constantemente busca bloques de memoria desreferenciados, que ya como tal no son accesibles por ningún medio, y por lo tanto estos ya no tienen impacto dentro del programa, pero que sin

embargo siguen ocupando memoria, y este está encargado entonces de liberarla. Java como tal no ofrece un método directo para interceptar el momento exacto en donde la memoria ocupada por un objeto en el heap es liberada. El método finalize(); se encuentra obsoleto, fue retirado por razones de seguridad y rendimiento, ya que nunca pudo ofrecer fiabilidad en su ejecución, por lo que la eliminación de un objeto quedó a cargo exclusivamente del run time dejando de lado el papel del programador.

Imaginemos el ciclo de vida de los objetos como un escenario en donde cada objeto es una persona. Cuando solicitamos la creación del objeto la persona sube al escenario, pero esta no tiene las luces enfocadas, simplemente está a oscuras, pero cuando las luces apuntan todas a este puede actuar y realizar su papel en la obra, interactuar con otros personajes, sin embargo, cuando su papel acaba, y las luces dejan de iluminarlo, este queda en la penumbra, inútil y sin función, ahí es cuando alguien del staff de la obra lo busca para sacarlo del escenario (el garbage collector).

Relaciones entre clases

Las relaciones entre clases constituyen la base para el modelamiento de un sistema orientado a objetos dentro de cualquier lenguaje, no solo en java, aunque este tiene, como todos los demás, sus particularidades. Realmente, aunque los objetos son autónomos en su estado y comportamiento, las relaciones entre estos permiten desarrollar funcionalidades más complejas que las que por sí solo un objeto podría ofrecer, potenciando así la capacidad de desarrollo para el programador. Comprender estas relaciones es necesario construir arquitecturas sólidas, reducir el acoplamiento innecesario (relaciones muy fuertes entre clases que son realmente innecesarias), y aumentar la modularidad dentro del código (que la responsabilidad de las funcionalidades esté repartida a muchas partes del código).

Asociación:

Es la relación más general y flexible que se puede establecer entre clases. Básicamente esta permite que un objeto tenga interacción con otro sin la necesidad de que la existencia de uno dependa fuertemente del otro, que no

exista la necesidad de que una sea parte de la otra, de forma que intercambien datos o se invoquen métodos. La relación de asociación puede ser unidireccional o bidireccional, dependiendo de si una clase conoce a la otra, o si ambas se conocen, ya que esta relación se suele simbolizar con atributos que hacen referencia a la otra clase.

Esta relación en resumidas cuentas nos permite que un objeto use las funcionalidades que ofrece el otro para lograr un fin específico, sin la necesidad de que la existencia de ese otro dependa exclusivamente del que lo use, sino que cada uno es independiente, pero se **asocian** para un fin particular. Se representa como "Tiene un", y nos sirve para cuando queremos establecer relación entre dos clases, mencionando que una o ambas poseen características de la otra

En UML se representa con una flecha uni-bi/direccional que va de la clase que es usada a la que usa.

Agregación:

La agregación es una forma especializada de asociación (un caso particular), la cuál permite modelar un objeto como un todo formado por partes que lo componen, el todo CONTIENE las partes, pero estas pueden existir de manera independientemente, es decir, como tal la clase que en este caso es un **todo** no controla la instanciación o el ciclo de vida de los objetos que la componen, y viceversa, simplemente esta los agrega como parte de si. Por ejemplo, un curso puede contener muchos estudiantes, pero los estudiantes existen independientemente del curso, o pueden pertenecer a más de un curso, y a su vez existen muchos cursos y no necesariamente todos deben tener los mismos estudiantes, sería una relación de muchos a muchos. Esta es una relación débil de pertenencia, pues a ambos objetos les importa poco si el otro existe o no.

La clase que contiene la llamamos huésped, y la contenida la llamamos invitado. El huésped puede acceder a las funcionalidades que el invitado expone mediante el API.
Usualmente la agregación se simboliza con colecciones que contienen objetos de otras clases, pero estos son independientes de la clase a la que componen.

En UML se representa con una línea que une el huésped con el invitado, y el huésped tiene un rombo vacío al final de la línea.

Composición:

La composición, por el contrario, establece una relación sumamente fuerte que implica control y responsabilidad integral sobre el ciclo de vida de una clase con respecto a la otra. Esta también es una relación todo-partes, en donde ahora el todo y las partes coexisten de manera que si uno es eliminado, el otro también, es decir, como tal una no tiene una razón de ser sin la otra. El ejemplo más clásico es el de una casa compuesta por habitaciones, si la casa deja de existir, las habitaciones no tienen como tal tampoco un papel razonable dentro del diseño. Usualmente se hace referencia a esta relación por medio de atributos que se declaran dentro de una clase, pero que al momento de construir el objeto no se pasan como parámetro al método constructor, sino que este internamente instancia el otro objeto haciendo un llamado a su constructor, por lo que no depende de valores externos sino de la existencia del todo mismo, si uno muere, el otro también, ya que una instancia es “dueña” de la otra, y controla su ciclo de vida

En UML se representa con una línea que va desde las subclases que componen el todo hasta el todo, y en vez de flecha un rombo lleno.

Dependencia:

Es la relación entre clases más trivial y débil. Está asociada más a métodos que a estructuras de clases como tal, ya que esta intenta reflejar que un objeto usa a otro para realizar una acción puntual, y luego de ello no existirá una relación de asociación entre ambos, y usualmente esta se suele representar como objetos que se requieren como parámetros para métodos de otro, o retornar un tipo de dato de otra clase, pero que luego de ejecutar esa acción no mantendrán una conexión fuerte a través del tiempo.

En UML se representa con una línea punteada con flecha que apunta a la clase que usa, la línea parte de la que es usada

---

Herencia:

La herencia permite crear clases en función de otras, permitiendo así establecer relaciones jerárquicas entre estas. Las clases del nivel jerárquico más alto son consideradas como las clases base, y las que son derivadas de estas se

consideran como casos particulares de las clases base, que pueden suplir la funcionalidad de los niveles de abstracción más altos, y también tener las suyas, es decir, conserva las invariantes de las clases base, pero también puede ampliar sus capacidades y funcionalidades ya sea creándolas o redefiniendo las heredadas. La herencia se representa de forma semántica como un " A es un B", mientras que por ejemplo la composición se representa como "A tiene un B". Gracias al principio de liskov podemos incluir el polimorfismo en esta ecuación, y hacer que los objetos de las clases hija puedan ser tratadas como un tipo más genérico (el de la clase padre) ya que suplen las necesidades de las clases base, y a parte poseen las suyas propias.

En UML se representa como una línea con flecha vacía apuntando al padre.

Ejemplo intuitivo de relaciones entre clases

Imaginemos las relaciones entre clases como un sistema de colaboración humana. La asociación podría ser una relación que podrías tener con otro humano en la cuál conversas de vez en cuando, la agregación podría ser la pertenencia a un grupo en particular, pero el grupo no te necesita necesariamente a ti, y tu puedes pertenecer a varios de estos, la composición podría verse reflejada en la relación con tus órganos, ya que estos no podrían vivir sin ti, y tu sin ellos, la herencia como una tradición heredada que puedes mantener o cambiar, y a parte crear las propias tuyas, y la dependencia se podría ver como realizar acciones que impliquen interactuar con otras personas, pero que no se mantenga con estas una relación más fuerte que trascienda su uso efímero.

Clases anidadas

En Java, el concepto de clase anidada surge como una necesidad en ciertas ocasiones para organizar de forma jerárquica la abstracción que se realizará a una entidad del mundo real. Básicamente, una clase anidada es una clase dentro de otra clase, pero esta definición se queda corta porque hay matices conceptuales sobre cuándo usarlo y qué tipo de clase anidada emplear.

En primer lugar, la necesidad de clases anidadas surge porque a veces la existencia de un ente específico hace que, de manera inherente, otro objeto tenga que nacer y sea razón de ser de esa clase; es decir, que no tiene sentido como

entidad individual dentro del sistema, sino que netamente hace parte de un objeto que, a su vez, forma parte del todo. El rango como tal de las clases anidadas se halla circunscrito dentro de la clase que las contiene; en otras palabras, su ámbito lógico está limitado al interior de la clase externa.

Java distingue entre dos tipos de clases anidadas: las clases anidadas internas y las clases anidadas estáticas. Sus diferencias no se deben precisamente a los modificadores que las componen, sino a los motivos de diseño y funcionalidad que cada una puede ofrecer, teniendo en cuenta la relación lógica que existe entre los objetos.

Clases anidadas internas

Las clases internas son un tipo de clases anidadas que mantienen una relación lógica bastante fuerte con la clase que las contiene. No solo se limita a tener una relación semántica con el objeto, sino que esta debe ir implícita dentro del ciclo de vida del objeto al cual compone, permitiéndosele así acceder también a las variables de instancia. Podemos imaginar este tipo de clases anidadas como una célula dentro del tejido: están ligadas directamente a este y se encuentran dentro de su ciclo de vida, componiendo una entidad común, el ser humano. Son parte de la estructura íntima de la clase externa. El hecho de que estas clases puedan acceder a las variables de instancia, incluso declaradas privadas, no significa en ningún caso la violación del encapsulamiento, sino la construcción de un encapsulamiento de nivel superior, el cual otorga cohesión al demostrar que ambas partes conforman un todo, cuya interfaz pública sigue siendo la de la clase externa. Además, estas clases guardan una referencia implícita a la clase que las creó.

Clases anidadas estáticas

Las clases anidadas estáticas son un tipo de clases anidadas que no mantienen una relación estricta, pero sí lógica, con la clase que las contiene. Tienen una razón de ser mucho más organizacional que de acoplamiento, ya que al ser declaradas como estáticas no dependen del ciclo de vida del objeto que las contiene internamente, sino que pueden ser funcionales sin necesidad de que la clase contenedora exista. Es decir, pueden ser instanciadas sin ningún tipo de problema, aunque no siempre se deba, pues muchas veces este tipo de clases

existen para organizar y aportar lógica auxiliar al objeto que las contiene; por ende, su constructor puede ocultarse. La clave está en la palabra *static* al momento de declararla: cuando la escribimos, la relación entre la clase contenedora y la contenida deja de ser una de instancias y pasa a ser una de tipos. Su utilidad radica entonces en representar una relación conceptual entre ambas entidades, que pueden ser independientes entre sí y no dependen del estado particular de las instancias. Esto permite tener un espacio de nombres más limpio y realizar abstracciones sin acoplamientos innecesarios, es decir, evitando que exista una relación más fuerte o más débil de lo que debería.

Para comprender el carácter teórico que impulsa a realizar un diseño con clases anidadas es necesario entender que, en ocasiones, los objetos se componen de otros, algo así como una relación todo-partes vista en las relaciones entre clases. Sin embargo, hay que matizar algo importante: esa relación solo se limita internamente al objeto. Es decir, el rango de las clases anidadas se ve limitado por la clase que las contiene; no obstante, esta puede ser expuesta por medio del API si así se desea. Lo importante es que estas no sean expuestas cuando su naturaleza es puramente íntima y sus responsabilidades no van más allá de la clase externa. Finalmente, su finalidad también es ayudar a la precisión semántica y a la correcta abstracción de la entidad.

REGLA DE ORO: si puedes hablar de un objeto sin la necesidad de hacer referencia a una clase contenedora, entonces ese objeto no puede ser modelado como una clase interna, porque su dominio va mucho más allá, su existencia si tiene un sentido lógico fuera de algo más que lo limite

---

Pass by value vs Pass by reference

El concepto de paso por valor está orientado al estudio de la acción de pasarle datos a los métodos para que estos puedan realizar su trabajo de manera efectiva. Sin embargo, en Java a lo largo de los años ha existido una confusión sobre el tipo de valores que se pasan a estos métodos. Como premisa fundamental debemos afirmar que en Java siempre se utiliza paso por valor y jamás paso por referencia; la idea de que Java pasa parámetros por referencia es producto de una lectura equivocada de cómo funciona el lenguaje.

En Java, cuando declaramos una variable que almacena un objeto, esta variable no contiene el objeto en sí, sino una referencia que apunta a la dirección en memoria donde ese objeto se encuentra. El objeto vive en el heap y la variable simplemente guarda una dirección en memoria o valor que indica su ubicación. El lenguaje no permite manipular direcciones de memoria de forma explícita, pero sí nos permite operar con estas referencias de manera abstracta.

Cuando un método recibe como argumento una variable que contiene una referencia, lo que se le pasa es una copia del valor de esa referencia. Esta copia no es "compartida" en el sentido de que exista una única referencia accesible desde dos lugares, sino que ambas variables —la externa y la del método— contienen valores idénticos que apuntan al mismo objeto. Por eso, tanto la variable original como el parámetro dentro del método pueden acceder al mismo objeto en memoria. Si el método modifica el estado interno del objeto al que apunta la referencia, ese cambio se reflejará también a través de la variable externa, porque el objeto es único y está siendo accedido desde dos referencias distintas pero equivalentes.

La situación cambia cuando el método intenta reasignar la referencia que recibió como parámetro. Si dentro del método se ejecuta una instrucción que hace que el parámetro apunte a un objeto diferente, esa reasignación afecta exclusivamente a la copia local de la referencia y nunca a la variable externa que se usó para invocar el método. La variable externa permanece apuntando al objeto original. El método, en cambio, termina apuntando a un objeto distinto porque su parámetro es simplemente un valor copiado al inicio de la invocación. En ese escenario pueden existir dos objetos distintos: el objeto original al que sigue apuntando la variable externa y un objeto nuevo al que apunta únicamente la referencia local dentro del método. Esta separación ilustra con claridad que Java no permite modificar la referencia contenida en la variable del invocador, sino únicamente el estado del objeto compartido

---

Static binding vs Dynamic binding

Cuando en Java una instrucción involucra la invocación de un método, el lenguaje debe determinar con exactitud qué cuerpo de método será ejecutado, lo cual requiere distinguir cuidadosamente entre dos mecanismos fundamentales: el

static binding y el dynamic binding. Esta distinción surge naturalmente de las dos formas clásicas de polimorfismo presentes en Java: el polimorfismo por sobrecarga y el polimorfismo por sobreescritura. Aunque ambos se relacionan con el uso de métodos que comparten nombre, la forma en que el lenguaje decide cuál método invocar difiere por completo en cada caso.

La sobrecarga de métodos, asociada al polimorfismo estático, corresponde a la coexistencia de varios métodos con el mismo nombre pero con distintas firmas. En Java, la firma de un método está compuesta por el nombre y la lista de parámetros, y es esta combinación la que permite al compilador seleccionar sin ambigüedad cuál versión del método debe ser invocada. El tipo de retorno no participa en la resolución de sobrecarga; el compilador solo analiza el nombre y los tipos de los argumentos proporcionados. Como esta información está completamente determinada antes de ejecutar el programa, el compilador puede identificar de manera inequívoca cuál método es el correcto incluso si múltiples métodos comparten el mismo nombre. Esta selección anticipada constituye un caso puro de static binding, en donde la decisión se cierra antes de la ejecución.

La situación es distinta en el caso de la sobreescritura, que corresponde al polimorfismo dinámico. Aquí interviene el principio de sustitución de Liskov, que establece que un objeto de una clase derivada debe poder ser usado en cualquier contexto donde se espere un objeto de la clase base. Esto implica que una variable declarada con un tipo general puede contener un objeto de un tipo más específico. En estos casos, si se invoca un método que ha sido sobrescrito en la subclase, la naturaleza del tipo declarado no basta para decidir qué implementación debe ejecutarse, porque el compilador únicamente conoce el tipo estático, el que se usa en la declaración de la variable. El compilador sabe que existe un método con ese nombre y esa firma, pero no puede determinar cuál implementación corresponde hasta conocer el tipo real del objeto.

Esta incertidumbre obliga a posponer la decisión hasta el tiempo de ejecución. Cuando la JVM encuentra una invocación de un método que puede estar sobrescrito, examina el tipo dinámico del objeto al que apunta la referencia. Ese tipo dinámico determina qué versión del método se ejecuta. Si el objeto es realmente una instancia de la subclase, se ejecuta la implementación sobrescrita; si no lo es, se invoca la implementación heredada. Este proceso es el dynamic binding, una resolución tardía que permite que el comportamiento del programa

dependa del tipo concreto del objeto en tiempo de ejecución y no únicamente del tipo declarado.

En consecuencia, el static binding y el dynamic binding representan dos momentos distintos de toma de decisión. En la sobrecarga, todo queda resuelto durante la compilación porque la selección depende exclusivamente de la firma compatible con los argumentos. En la sobreescritura, por el contrario, el compilador no puede decidir en ausencia del tipo dinámico del objeto, de modo que la JVM toma la decisión en tiempo de ejecución. Esta diferencia explica por qué el polimorfismo dinámico es un rasgo esencial de la programación orientada a objetos y por qué Java sostiene este modelo basándose en el Liskov Substitution Principle.

Records

Los records representan una de las innovaciones más profundas en la evolución moderna de Java, comparables en impacto al advenimiento de las expresiones lambda y las APIs funcionales introducidas en Java 8. Su aparición responde a una necesidad histórica en la programación orientada a objetos: no todas las abstracciones del dominio requieren comportamientos complejos ni un ciclo de vida mutable. En muchos escenarios el valor esencial de una entidad radica en sus atributos fundamentales y no en la lógica que pueda encapsular. Dinero, direcciones, documentos de identidad, etc, suelen funcionar, en esencia, como unidades de información cuyo papel principal es transportar un conjunto coherente de datos, más que ejecutar comportamientos elaborados y a parte no deben cambiar.

Antes de los records, modelar tales entidades como clases suponía una carga sintáctica considerable. La herencia desde `Object` obliga a proporcionar implementaciones coherentes de métodos como `equals`, `hashCode`, `toString` y `canEqual`. A esto se sumaba la necesidad de declarar manualmente campos privados, métodos de acceso, constructores y, en ocasiones, `setters`, aunque conceptualmente la entidad fuera naturalmente inmutable. Todo este trabajo, repetitivo y de poco valor conceptual, introducía ruido en el código y aumentaba el riesgo de inconsistencias sutiles en la comparación y el manejo de objetos.

Los records surgen precisamente para resolver este problema. Al igual que las clases, permiten representar abstracciones del mundo real, pero orientan su semántica hacia la noción de identidad basada en valores. Un record define su esencia mediante sus componentes, que el lenguaje interpreta como el estado lógico y completo del objeto. A partir de esta definición mínima, el compilador genera automáticamente campos privados y finales, métodos accesores, un constructor canónico capaz de asignar los componentes, implementaciones correctas y consistentes de los métodos heredados de `Object` y una representación textual adecuada. Esta automatización permite que el programador se concentre en el modelo conceptual sin distraerse con detalles mecánicos.

La inmutabilidad que proporcionan los records debe entenderse con precisión. Los campos declarados dentro del record son finales, de modo que la referencia a los objetos que estos contienen es inmutable. Sin embargo, ello no implica necesariamente que el contenido de esos objetos lo sea. La inmutabilidad garantizada por el record es superficial o estructural: asegura que los componentes del record no pueden ser reasignados, pero no impone restricciones sobre la mutabilidad interna de los objetos referenciados. Si los componentes son instancias de tipos inmutables, entonces la identidad del record es completamente estable. Si, en cambio, los componentes son estructuras mutables, su modificación interna puede alterar de manera indirecta el estado lógico del record, a pesar de que sus referencias permanezcan constantes. Por esta razón, en contextos donde la inmutabilidad profunda es deseada, resulta recomendable utilizar tipos inmutables o realizar copias defensivas de los objetos mutables que se reciben en el constructor.

En cuanto a su posición dentro del sistema de tipos de Java, los records no pueden participar en jerarquías de herencia tradicionales. El lenguaje los declara implícitamente como `final`, de modo que no pueden tener subclases ni extender otras clases, salvo de manera automática la clase `Object`. Esta restricción no es arbitraria: la identidad basada en valores exige que la estructura del record permanezca fija y que no pueda ser alterada mediante extensiones que añadan o modifiquen componentes. No obstante, los records sí pueden implementar interfaces, ya que estas no afectan la estructura lógica del objeto, sino únicamente su comportamiento contractual. Este mecanismo permite que los

records mantengan una función esencialmente estructural, sin perder la capacidad de integrarse en modelos polimórficos fundamentados en interfaces

El nombre del archivo .java debe ser igual que el nombre del record, de manera análoga a como sucede con las clases

Además, los records pueden mantener relaciones entre clases (menos de herencia) y tener sus propios métodos; la diferencia radica en la inmutabilidad de los objetos que se pueden diseñar con este tipo de clases

---

Initializer Blocks

En java la construcción de un objeto se diferencia de la inicialización de sus atributos; cuando un objeto se construye se reserva memoria en el heap y se asignan valores predeterminados para cada atributo, cuando el objeto se inicializa se le dan valores particulares a cada atributo dependiendo del contexto. Usualmente esta lógica de asignación o inicialización suele corresponder al método constructor, sin embargo hay que reconocer que esto no siempre aplica, y que hay lógica que debe ser ejecutada incluso antes de que el constructor sea llamado o que este proceso sea compartido por múltiples constructores

Existen dos tipos de bloques de inicialización:

- Instance Initializer blocks: Es un bloque de lógica de inicialización que se ejecuta cada que un objeto se crea previo al llamado del constructor, funciona cuando múltiples constructores requieren implementar obligatoriamente la misma operación, sin embargo no podemos dejar esta responsabilidad a cargo de otro método privado ya que esto mezcla etapas del ciclo de vida del objeto, la construcción con el periodo de vida útil, entonces lo correcto es usar estos bloques para que la lógica se ejecute y mantener así un estado válido del objeto en todo momento y no comprometer su integridad a futuro, por ejemplo invocando ese método privado en otra parte del programa
- Static initializer blocks: Este bloque de lógica al ser estático se ejecuta una sola vez cuando la clase se carga en memoria, y permite ejecutar bloques de código que permiten configurar de forma adecuada la clase antes de cualquier instanciación de la misma, este bloque precede a las demás

operaciones y permite plasmar un punto de partida válido para el ciclo de vida de los objetos

Los bloques de inicialización se distinguen del constructor porque pertenecen a niveles de abstracción distintos. Mientras que el constructor se centra en una instancia particular, el instance initializer block se centra en todas las instancias, y en el caso del static, a la clase en sí. Estos bloques son más cercanos a la definición estructural de la clase que a la particularización o materialización de esta en una instancia

---

Enums

Los enums en java son algo especiales, ya que a diferencia de otros lenguajes, estos no son solo una especie de diccionario con claves y valores. En java, los enums se tratan como clases especializadas, las cuales permiten almacenar de forma vectorial (según posiciones) y consciente, valores previamente determinados según la lógica de negocio, cada constante realmente es una única instancia inmutable de la clase del tipo que estamos creando, por lo que si intentamos cambiar la referencia o trabajar con un valor que no existe, habrá un error en tiempo de compilación. Es extremadamente útil cuando se conocen de antemano todos los valores posibles que puede tomar una variable de cierto tipo, acá algunos ejemplos:

- Los días de la semana: Lunes, Martes, Miércoles, Jueves, Viernes, Sábado, Domingo
- Puntos cardinales
- Lista de rangos o privilegios de una empresa o sistema
- etc

Los enums permiten tener un mayor control sobre el sistema, y se pueden aplicar a sistemas compuestos, en donde por ejemplo, el tipo de dato tenga ciertas características, y métodos para trabajar con el, pero que como mencioné anteriormente, de antemano se sepa todos los valores que puede tomar el objeto.

Encadenamiento de métodos

El encadenamiento de métodos es una técnica muy usada que provee java para poder llamar en una misma línea de código a varios métodos sin la necesidad de hacer múltiples llamados para una misma instancia, esto es gracias a un pequeño truco que se logra con el retorno en cada uno de los métodos el tipo de dato de la clase, en lugar de otro completamente distinto, lo que provee la funcionalidad de inmediatamente poder llamar a otro método de la clase, ya que una instancia de esta lo está llamando, lo cuál es válido

```
Objeto obj = new Objeto();
obj.correr();
obj.saltar();
obj.bailar();
```

En este ejemplo estamos llamando en cada línea distinta a un método distinto para el objeto en cuestión, y supongamos que estos métodos están declarados dentro de la clase como void.

Ahora bien, para poder hacer encadenamiento de métodos el truco está en tener como tipo de retorno el de la clase, y retornar el estado de la instancia actual `this`

```
public class Objeto {
    /*
    Atributos
    .
    .
    .
    */

    //--Constructor--
    public Objeto correr() {
        //logica
        return this
    }
    public Objeto saltar() {
        //logica
        return this
```

```
    }
    public Objeto bailar() {
        //logica
        return this
    }
}
```

Así podremos realizar lo siguiente

```
Objeto obj = new Objeto();
obj.correr().saltar().bailar();
```

Esto funciona ya que los métodos se ejecutan de manera secuencial, y cada vez que uno termina su que hacer realizando los cambios correspondientes en lainstancia, retorna el estado actual de la misma, y permite que de una vez el siguiente la reciba ya que es del tipo de dato de la clase para que este realice su tarea, y así sucesivamente hasta realizar todas y cada una de las acciones que corresponden a los llamados.

La Inmutabilidad

La inmutabilidad hace referencia a que una vez un objeto sea creado dentro del programa su estado interno no puede cambiar, previniendo así fallos derivados de, por ejemplo, la programación multihilo, o cambio de atributos que puedan comprometer la integridad interna del objeto en cuestión.

Para que una clase sea completamente inmutable, podemos declararla con la palabra `final`, para que así esta no pueda pertenecer a cadenas de herencia (parecido a los métodos finales), también, podemos declarar sus atributos como finales, para que la referencia o valor no pueda ser cambiado una vez se asigna, y no exponer ningún tipo de setter para así definitivamente privar el acceso una vez se creó el objeto, y siempre devolver copias de objetos mutables dentro de los métodos, no la referencia exacta

El buen uso de la palabra final es crucial para un diseño meticuloso dentro del software, esta también puede usarse dentro de los parámetros de métodos para

evitar una reasignación accidental del valor dentro del cuerpo del método, TODO LO QUE PUEDA SER FINAL DEBE SER FINAL